Autonomous Taxi Control via Reward Design

Authors: Marwan Zayyad – 208264283 Baraah Dahle – 324888551

Course: Advanced Reinforcement Learning

Environment: Gymnasium Taxi-v4

1. AI Use Disclaimer

Artificial Intelligence tools, primarily **ChatGPT (OpenAI)**, were used only as supporting development tools throughout this project. AI assistance was limited to tasks such as clarifying reinforcement learning concepts, explaining documentation, helping resolve programming bugs, improving code readability, and refining the writing and organization of the final report.

AI was mainly used to clarify reinforcement learning concepts encountered during development, explain unfamiliar APIs and library functions, help identify and resolve programming bugs, review pieces of code when unexpected behavior occurred, suggest cleaner implementations for existing code, and improve the clarity, grammar, and organization of the final report.

In summary, AI served as a development assistant that accelerated routine programming and documentation tasks, while the core software engineering, reinforcement learning implementation, experimental design, and evaluation remained entirely the responsibility of the authors.

*** All implementation decisions, experiment planning, reward function design, hyperparameter selection, algorithm integration, debugging strategy, execution of the experiments, analysis of the obtained results, and conclusions drawn from those results were performed by us. Whenever alternative solutions or implementation approaches were considered, the final decisions were made based on our own testing, experimental observations, and engineering judgment.***

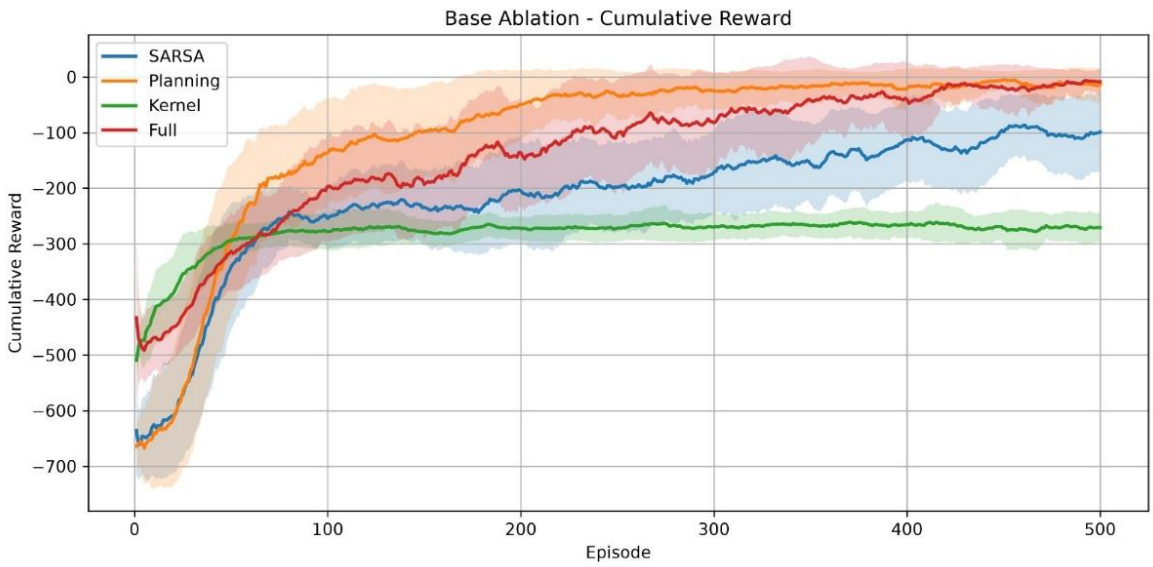
2. Beyond the Minimum Requirements

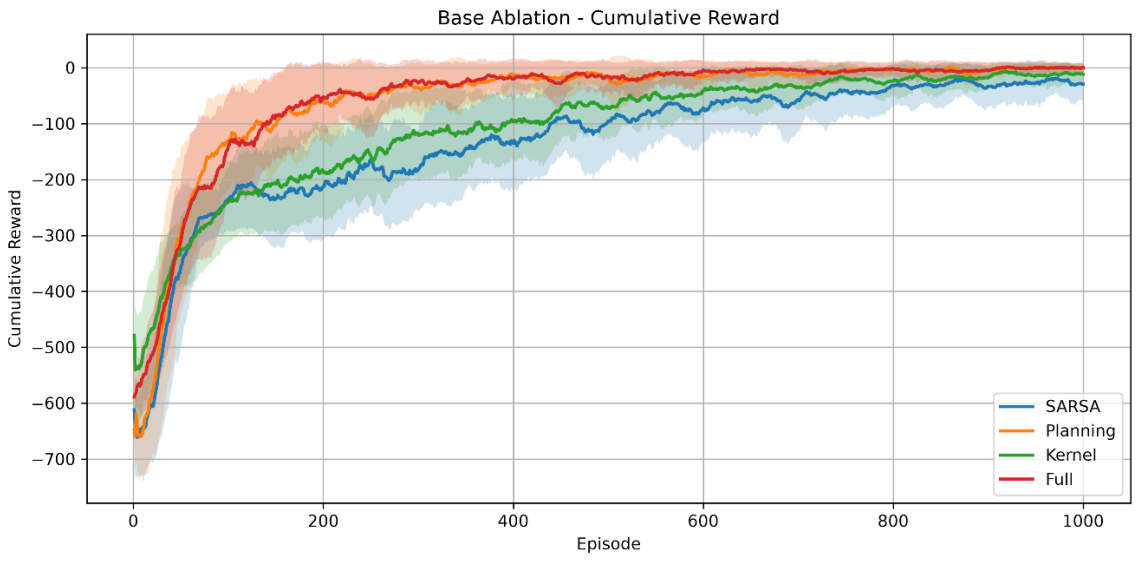
In addition to completing all the mandatory project requirements, we extended the project in several ways to provide a more comprehensive experimental study:

- **Implemented five reward functions** instead of the required four, allowing a broader comparison between different reward design strategies.

- **Evaluated six performance metrics** (Cumulative Reward, Episode Length, Success Rate, Illegal Actions, Mean TD Error, and Unique States Visited), during development to analyze agent behavior and identify the most informative metrics. The final report presents the two metrics that provided the clearest comparison between methods.
- **Performed extensive hyperparameter tuning**, evaluating multiple values of **α** and **planning steps**. After the initial experiments produced unsatisfactory results, additional tuning of **ε** was performed to improve learning stability and overall performance.
- **Conducted iterative kernel optimization**, evaluating **5, 10, 15, and 20 kernel neighbors**, as well as **5 and 10 planning steps**, and testing different kernel parameter settings before selecting the final configuration that achieved the best overall performance.

Example: Kernel Parameter Tuning before (top) and after (bottom) tuning the kernel parameters and neighborhood size.

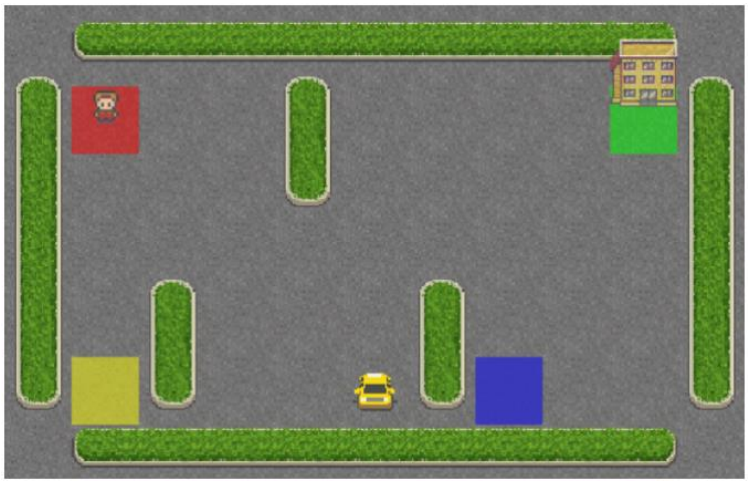




3. Environment / Map Used for Training

The **Taxi-v4** environment is a widely used reinforcement learning benchmark that includes key challenges such as sequential decision-making, navigation, exploration, planning, and delayed rewards. It consists of a **5 × 5 grid world** with four predefined colored locations. At the beginning of each episode, the taxi is randomly placed on the grid, while a passenger appears at one of the four locations and must be transported to a different destination. The objective of the agent is to reach the passenger, pick them up, navigate to the destination, and successfully drop them off. Internal walls block movement between certain cells, so the optimal path is not always the shortest Manhattan path. This requires the agent to learn valid navigation routes through exploration.

All experiments in this project were performed using the default **Taxi-v4** environment provided by **Gymnasium** with fixed random seeds to ensure reproducibility.



State space

The Taxi-v4 environment contains **500 discrete states**. Each state is defined by three variables:

- The taxi's position (**25** possible grid locations)
- The passenger's location (**5** possibilities, including inside the taxi)
- The destination (**4** possible locations)

This results in:

25 × 5 × 4 = 500 possible states.

Despite its moderate size, the state space is sufficiently large to require efficient exploration and planning for effective learning.

Action space

The agent can perform six discrete actions:

Action	Description
South	Move one cell south
North	Move one cell north
East	Move one cell east
West	Move one cell west
Pickup	Pick up the passenger
Dropoff	Drop off the passenger

4. Reward Design

A primary objective of this project was to investigate how different reward functions influence the learning process of a reinforcement learning agent. Besides the default Taxi-v4 reward and a sparse reward baseline, three custom reward functions were designed to encourage different behaviors. Each reward was motivated by a different learning intuition and evaluated experimentally.

4.1 Fast Delivery Reward

The Fast Delivery Reward encourages the agent to complete each task as efficiently as possible. In addition to the standard environment rewards, a dynamic completion bonus is awarded based on the number of steps required to finish the episode. Faster task completion therefore produces a higher total return.

The intuition behind this design is that rewarding shorter solutions encourages the agent to minimize unnecessary movement and discover efficient navigation strategies.

- **Successful drop-off:** +20 + **max(0, 20 − step_count/10)**
- **Each time step:** −1
- **Illegal pickup/drop-off:** −10

4.2 Safe Driving Reward

The Safe Driving Reward emphasizes avoiding costly mistakes during navigation. Illegal pickup and drop-off actions receive a stronger penalty than in the default environment, while collisions with walls are also penalized. Successful task completion receives a larger terminal reward.

The intuition is that discouraging invalid and inefficient actions should produce smoother navigation behavior and reduce unnecessary exploration.

- **Successful drop-off:** +30
- **Each time step:** −1
- **Illegal pickup/drop-off:** −15
- **Wall collision:** −3

4.3 Progress Shaping Reward

The Progress Shaping Reward provides continuous feedback throughout the episode. Instead of rewarding only successful task completion, the reward is determined by whether the taxi moves closer to or farther from its current objective, measured using Manhattan distance.

This reward shaping reduces the delayed reward problem by providing informative feedback after nearly every action, allowing the agent to identify beneficial decisions much earlier during training.

- **Move closer to the current objective:** +1
- **Move farther from the current objective:** −2
- **No progress:** −1
- **Illegal pickup/drop-off:** −10
- **Successful drop-off:** +30

4.4 Kernel-Based Generalization

Traditional tabular reinforcement learning treats every state independently, meaning experience gained in one state is not transferred to similar states. To overcome this limitation, we implemented a **Continuous Geometrical Kernel** that generalizes each learning update to nearby states.

Similarity is computed only between states that share the same passenger status and destination. Among these states, the **five nearest neighbors** (selected using Manhattan distance) receive a weighted update based on the kernel similarity function

$$K(s, s') = \frac{1}{1 + e^{\beta(d_{Man}(s,s') - c)}}$$

where:

- $d_{Man}(s, s')$is the Manhattan distance between the taxi positions.
- βcontrols how quickly similarity decreases with distance.
- $c$determines the distance at which the decay begins.

As a result, updates are propagated not only to the visited state but also to nearby task-equivalent states. The intuition is that similar states should exhibit similar value estimates, allowing useful experience to be shared and accelerating the learning process.

5. Summary of Findings

The experiments show that both **reward design** and **algorithmic components** strongly influenced the learning performance of the SARSA agent in the Taxi-v4 environment. However, the most important finding was that a better reward signal often improved learning more than simply adding more computation.

The reward design process was iterative. The first custom reward, which encouraged faster delivery, improved performance moderately by pushing the agent toward shorter episodes. The second custom reward, which emphasized safe driving by increasing penalties, performed worse than expected. This showed that stronger punishment does not automatically lead to better learning, because excessive penalties can make exploration less useful and slow down policy improvement.

This observation motivated the third reward design, the **Progress Shaping Reward**. Unlike the previous rewards, it provided feedback throughout the episode by rewarding movement toward the current objective and penalizing movement away from it. This made the learning signal much more informative and reduced the delayed reward problem. As a result, the agent learned more stable and efficient behavior.

The ablation experiments showed that **Prioritized Sweeping** was the most important algorithmic extension. By reusing previously observed transitions, planning allowed the agent to learn more from each real environment interaction. The kernel-based generalization also improved learning after tuning, but its benefit depended more strongly on parameter selection. Early kernel configurations performed poorly, while the final tuned version showed that local value propagation can help when similarity is defined carefully.

The parameter sweeps further showed that moderate parameter choices were generally better than extreme ones. A smaller and stable learning rate produced smoother convergence, while a balanced exploration rate gave the agent enough exploration without preventing exploitation. Similarly, increasing planning or kernel neighborhood size did not always improve performance; after a certain point, the extra updates added computation and sometimes spread information too broadly.

Overall, the best behavior was achieved when **informative reward design**, **Prioritized Sweeping**, and **Kernel-based Generalization** were combined. The reward function improved the quality of the learning signal, planning improved sample efficiency, and the kernel helped transfer useful updates between similar states. Together, these components produced faster convergence, higher rewards, and shorter final trajectories than using SARSA alone.

Detailed plots, exact parameter comparisons, and deeper analysis are presented in the Results section.

6. Experiment Setup and Algorithm Specification

All experiments were performed using the **Gymnasium Taxi-v4** environment with a fixed random seed (**42**). Each agent was trained for **1000 episodes**, with a maximum of **200 steps per episode**. Results were averaged over **5 random seeds**, and the final policies were evaluated over **100 episodes**.

Parameter	Value	Description
α	0.1	SARSA learning rate controlling the size of each Q-value update.
γ	0.95	SARSA discount factor determining the importance of future rewards.
ε	0.1	ε-greedy exploration rate used for action selection in SARSA and planning.
Planning Steps (n)	5	Number of Prioritized Sweeping updates after each real interaction.
β	4.0	Kernel parameter controlling how quickly similarity decreases with distance.
c	0.5	Kernel distance threshold defining the neighborhood of similar states.
Kernel Neighbors	5	Maximum number of task-equivalent neighboring states updated by the kernel.

7. Best Trained Policy Animations

To further evaluate the learned policies, we generated Gymnasium animations of the trained agents after training was completed. The animations were created by loading the learned Q-tables and executing new episodes using the final policies, allowing the agent's behavior to be visualized independently of the training process.

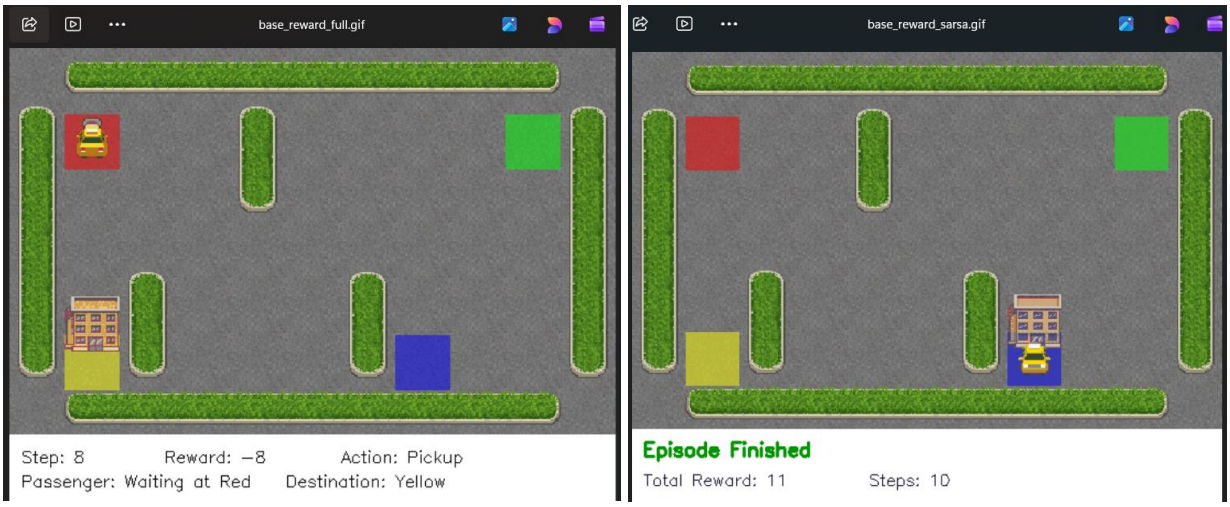
The generated animations include:

- A comparison between the three individual approaches (**SARSA**, **Prioritized Sweeping**, and **Kernel-based Generalization**) using the Base Reward.
- A policy evolution animation of the complete algorithm (SARSA + Prioritized Sweeping + Kernel-based Generalization), illustrating how the agent improves throughout training and comparing its final behavior with earlier training stages.

To create the policy evolution animation, the training process was first analyzed to identify representative milestones where noticeable improvements occurred. The project was then re-executed while saving the policy, initial state, and executed actions at those selected episodes. These checkpoints were later replayed to produce a chronological visualization of the agent's learning progress.

**** Since animated GIFs cannot be embedded directly into the PDF report, the resulting animations are provided in the accompanying presentation or it can be generated again using the instructions (in the final part of the report).****

Snapshots from the Generated Animations:



8. Surprising Results

During development, several unexpected behaviors were observed:

- **A denser kernel did not improve performance.** Increasing the number of neighboring states (e.g., from 5 to 20) unexpectedly reduced performance. Although more information was propagated, many updates reached less relevant states, slowing convergence and reducing policy quality.
- **More planning was not always better.** Increasing the number of planning steps beyond 5 produced only marginal improvements while noticeably increasing computation time. This demonstrated that excessive planning can lead to diminishing returns.
- **The best-performing reward was not our first design.** Our initial reward functions produced only minor improvements, and one even performed worse than the original Taxi reward. This motivated the design of the Progress Reward, which provided continuous guidance and resulted in significantly better learning performance.
- **Longer training became necessary after optimization.** Initially, 500 training episodes appeared sufficient. However, after improving the planning and kernel components, the agent was still learning effectively at episode 500. Increasing training to 1000 episodes allowed the policy to fully converge and achieve its best performance.

9. Exploration vs. Exploitation Across the Reward Designs

The exploration–exploitation trade-off was controlled using an **ε-greedy** policy with **ε = 0.1**, allowing the agent to exploit the current best action most of the time while still exploring random actions to discover potentially better strategies.

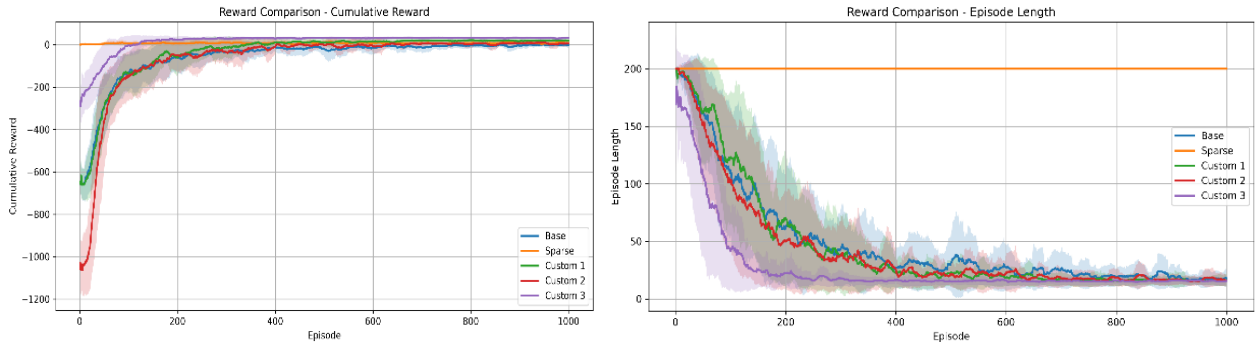
The reward design had a significant impact on this trade-off. With the **base reward**, the agent received limited guidance before completing the task, requiring more exploration to identify successful behaviors. The first two custom reward designs slightly changed this behavior, but did not consistently improve learning efficiency.

In contrast, the **Progress Reward (Custom Reward 3)** provided continuous feedback by rewarding movements toward the current objective. This made informative actions easier to identify, allowing the agent to exploit successful behaviors earlier while still maintaining sufficient exploration through the ε-greedy policy. As a result, learning became faster and more stable compared to the other reward designs.

10. Results & Plots

10.1 Rewards Comparison

To evaluate our reward engineering approach, we compared the baseline reward with four alternative reward designs. The results show cumulative reward during training and average episode length over time, highlighting differences in learning performance and efficiency.



Cumulative Reward Analysis

The cumulative reward curves show clear differences in learning performance across the reward functions. **Progress Shaping (Custom Reward 3)** achieved the fastest improvement and highest cumulative rewards by providing continuous feedback throughout each episode. **Fast Delivery (Custom Reward 1)** also improved over the baseline by encouraging shorter and more efficient trajectories. In contrast, **Safe Driving (Custom Reward 2)** learned more slowly due to stronger penalties that reduced early exploration, while **Sparse Reward** performed worst because the limited feedback forced the agent to rely heavily on random exploration.

Overall, denser and more informative reward signals significantly improved learning speed and convergence.

Episode Length Analysis

The episode length results closely matched the cumulative reward trends. Initially, all agents required nearly **200 steps** per episode, reflecting near-random behavior. As learning progressed, the more effective reward functions rapidly reduced episode length, demonstrating improved navigation efficiency.

Progress Shaping reached near-optimal episode lengths the fastest, followed by **Fast Delivery**. **Safe Driving** converged more gradually but eventually produced stable behavior, whereas **Sparse Reward** remained close to the maximum episode length for much of training due to the lack of informative intermediate feedback.

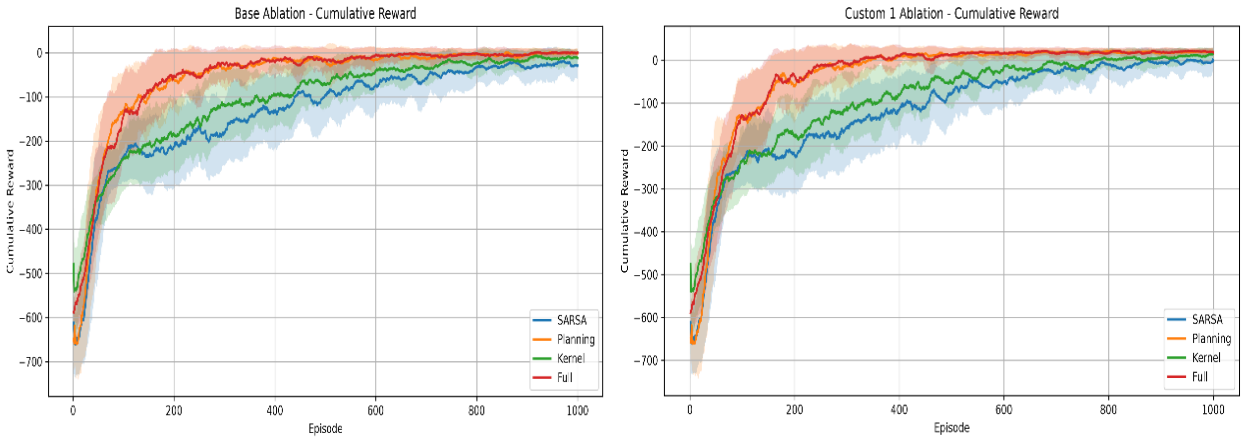
Discussion

The experiments demonstrate that **reward design has a major influence on learning efficiency**. Although all reward functions target the same task, the amount and quality of feedback strongly affect how quickly the agent learns.

Dense reward signals reduce the delayed reward and credit assignment problems, allowing the agent to identify beneficial actions much earlier. In contrast, sparse rewards provide little guidance and slow convergence. Overall, **Progress Shaping (Custom Reward 3)** achieved the best performance by combining continuous feedback with stable learning, resulting in faster convergence, higher cumulative rewards, and shorter episodes than all other reward functions.

10.2. Ablation Study: Cumulative Reward Comparison

This section evaluates the contribution of each component under the Base reward and Custom Reward 1 (Fast Delivery) by comparing cumulative reward across training episodes, as shown in the plots:



The following four configurations were evaluated: SARSA (baseline), SARSA with Planning, SARSA with a Geometrical Kernel, and a full model combining both Planning and the Kernel. Experiments

were conducted under both the original Taxi reward and the Fast Delivery reward to ensure consistent evaluation across different reward settings.

Overall Trend Across Both Rewards : Across both plots, a clear and consistent ranking is observed. The Full model performs best, achieving the fastest convergence and most stable learning. Removing components progressively reduces performance, with SARSA showing the slowest learning. The ranking remains consistent across both reward settings: **SARSA < Planning < Kernel < Full Model**.

Base Reward Analysis:
Under the Base Taxi-v4 reward configuration in the left plot, the system is highly sensitive to the removal of core components:

- **SARSA (Blue):** This baseline serves as the lowest bound. Without supplemental components, value propagation is sluggish, lagging horizontally and showing substantial performance volatility before slowly climbing toward a sub-zero baseline by episode 800.
- **Planning (Orange) & Kernel (Green):** Incorporating planning or kernel mechanisms provides a noticeable boost over standalone SARSA. The Planning component significantly accelerates early learning up to episode 200, while the Kernel variant helps bridge late-stage performance gaps, though both still fall short of the optimal threshold.
- **Full (Red):** The complete model dominates from the early training phases, rising sharply out of the low-reward valley (below -600) and stabilizing securely near 0 by episode 300.

  Insight: The sparse nature of the base reward highlights why a combined approach is necessary. Standalone SARSA struggles to update distant states effectively, whereas integrating planning and kernel methods provides the necessary sample efficiency to maximize returns rapidly.

Fast Delivery Reward Analysis:
Under the Custom Reward (Fast Delivery) setting shown in the right plot:

- **Denser rewards lift all configurations:** The learning curves are slightly more compressed vertically during the ascent, as the shaped reward feedback guides even the basic SARSA baseline more effectively.
- **Persistent performance gaps:** Despite the helper signals, the Full model (red curve) still secures the absolute highest returns with the fastest stabilization timeline.
- **Planning vs. Kernel dynamics:** The Planning baseline (orange) shows an exceptionally strong early performance matching the Full model's ascent rate for the first 100 episodes, but fails to maintain the same final asymptotic stability as the Full configuration.

Key Difference: Dense rewards improve convergence for all methods, but the Full model still achieves the fastest, most stable, and most robust learning.

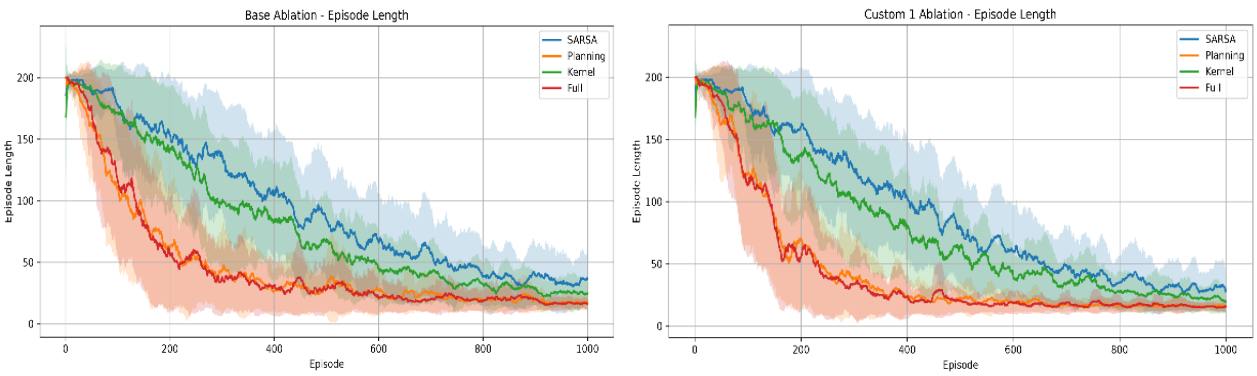
Key Insights :

- The Full model performs best, showing the benefit of combining Planning and the Kernel.
- Planning provides the largest improvement in early learning speed.
- The Kernel improves stability by smoothing value updates across similar states.

Conclusion: The ablation study confirms that each component contributes to performance, but combining both yields the fastest, most stable convergence and the highest cumulative rewards across both reward settings.

10.3 Ablation Study: Episode Length Comparison

To evaluate each component, four variants were compared: SARSA (baseline), SARSA with Planning, SARSA with a Geometrical Kernel, and the Full model combining both. Episode length was used as the main metric, where shorter episodes indicate more efficient navigation and higher policy quality.



Base Reward Function
The right plot shows learning curves under the original Taxi-v4 reward. All methods start near the maximum episode length (200 steps), indicating initial random behavior.

SARSA improves slowly due to reliance on real transitions only. Adding Planning significantly accelerates learning by propagating updates through simulated experiences. The Kernel-only variant also improves performance via state generalization but converges more slowly than Planning.

The Full model performs best, achieving the fastest convergence and lowest episode lengths by combining Planning and Kernel-based generalization.

Custom1 Reward Function
The left plot shows the same comparison under the Custom Reward. The overall behavior remains consistent, with the same ranking: Full > Planning > Kernel > SARSA.

This indicates that Planning and the Geometrical Kernel provide robust improvements independent of reward design. The Full model also converges slightly more smoothly, suggesting that reward shaping enhances the combined effect of both components and speeds up learning.

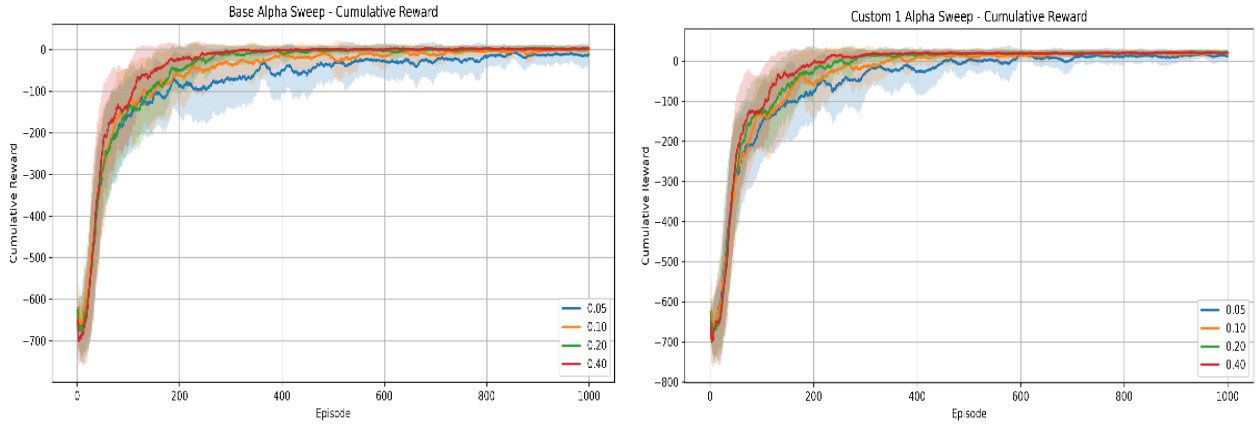
Variability During Training : The shaded regions represent variance across different random seeds. Early training shows high variance for all methods due to random exploration, but it decreases as policies stabilize.The Full algorithm achieves both the lowest episode length and the smallest variance after convergence, indicating more consistent and stable performance across runs.

Discussion: The ablation study shows that Planning provides the largest improvement by increasing sample efficiency through simulated experience. The Geometrical Kernel further improves learning by sharing information between similar states, reducing redundant exploration and speeding up value propagation.

Combining both yields the best results, as Planning improves temporal efficiency through replayed experience, while the Kernel improves spatial generalization. Together, they produce faster convergence, shorter episodes, and lower variance.

10.4 Hyperparameter (α) Analysis: Cumulative Reward Comparison

This section examines the impact of the learning rate α on performance under both the base reward and Custom Reward 1 settings. Results are presented using cumulative rewards across training episodes for different α values.



Overall Trend Across Both Rewards : Across both plots, a consistent pattern is observed: very low α values (0.05–0.10) result in slow learning and delayed convergence, while medium values (0.20–0.40) achieve faster convergence and higher final returns. Higher α values improve early learning speed but may introduce slightly more instability in the initial episodes. Overall, the learning rate strongly influences both stability and convergence speed regardless of reward design.

Base Reward Analysis (right Plot)

Under the Base Taxi-v4 reward, performance is highly sensitive to the learning rate α. Very low values (0.05–0.10) lead to slow learning and delayed convergence, as the agent struggles to effectively propagate sparse reward signals through the state space. In contrast, moderate values (0.20–0.40) perform best, achieving faster convergence and higher cumulative rewards by balancing stability with efficient adaptation. Overall, the sparse reward structure requires a sufficiently large α to enable effective Q-value propagation.

Custom Reward 1 Analysis (left Plot)

Under the **Custom Reward (Fast Delivery)** setting:

- Learning is **faster and more stable overall** compared to the base reward.
- Even lower α values improve due to denser reward feedback.

- However, **α ≈ 0.20–0.40 still performs best**, reaching the highest returns.

Key difference: Because the reward is denser, the system is less sensitive to α, but optimal performance still occurs in the same mid-range region.

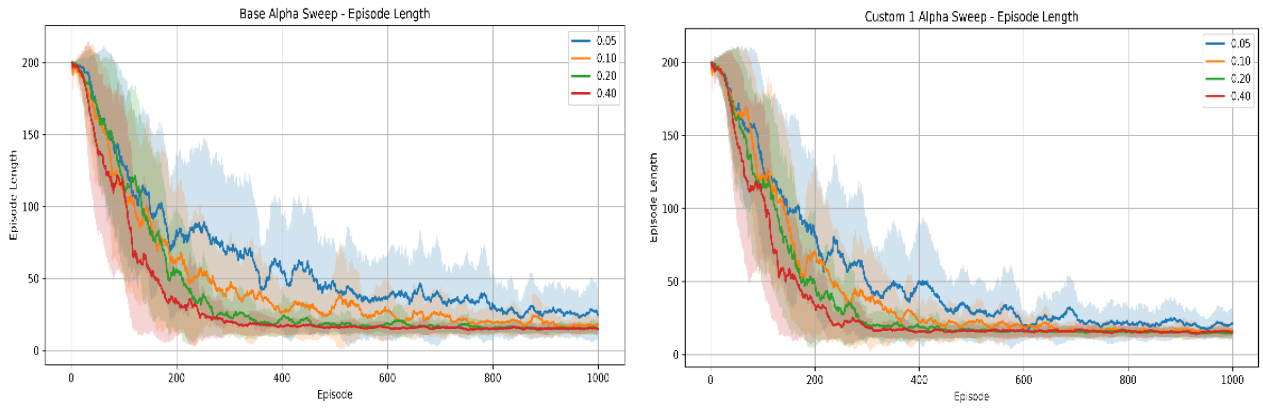
Key Insights :

1. Optimal α range: 0.20–0.40
   This range provides the best balance between fast learning and stability across both reward settings.
2. Low α slows convergence significantly
   Small learning rates make the agent learn too cautiously, limiting effective value propagation.
3. Higher α speeds up learning but may reduce stability
   Large values improve early progress but can introduce instability, especially in sparse reward environments.
4. Reward structure affects sensitivity to α
   Sparse rewards (Base) require more careful tuning of α than denser, shaped rewards (Custom 1).

Conclusion : The results show that α is a key parameter controlling the trade-off between stability and learning efficiency. Across both reward settings, mid-range values (0.20–0.40) consistently perform best, offering the most effective balance between convergence speed, stability, and exploration.

10.5 Hyperparameter (α) Analysis: Episode Length Comparison

This section examines the impact of the learning rate α on the agent's operational efficiency under both the Base reward and Custom Reward 1 settings. Results are presented using episode length across training episodes for different α values.



Overall Trend Across Both Rewards

Across both plots, a consistent pattern is observed. Low learning rates (α = 0.05–0.10) slow down learning, causing gradual reductions in episode length and delayed convergence. In contrast,

higher values (especially α = 0.40) produce a rapid drop in episode length within the first ~200 episodes, indicating faster learning of efficient paths. Overall, α consistently affects learning speed across both reward settings, independent of reward design.

Base Reward Analysis (right Plot)
Under the Base reward configuration, episode length is highly sensitive to the learning rate α. With α = 0.05–0.10, learning is slow, and episode length decreases gradually while remaining unstable, without fully converging even after 1000 episodes. In contrast, α = 0.20–0.40 leads to much faster improvement, with α = 0.40 achieving the quickest reduction from the 200-step cap to a stable range of ~15–20 steps early in training. Overall, the sparse reward structure requires a relatively high α to efficiently reduce long exploratory trajectories, while small learning rates lead to prolonged and inefficient episodes.

Custom Reward 1 Analysis (left Plot)
Under the Custom Reward 1 (Fast Delivery) setting, the agent shows faster and more stable convergence than under the Base reward. Episode length decreases more smoothly, indicating that denser feedback improves path compression.

The α = 0.40 configuration performs best, achieving the steepest and fastest reduction in episode length. Lower values such as α = 0.05 learn more slowly, but the denser reward reduces variance compared to the Base case.

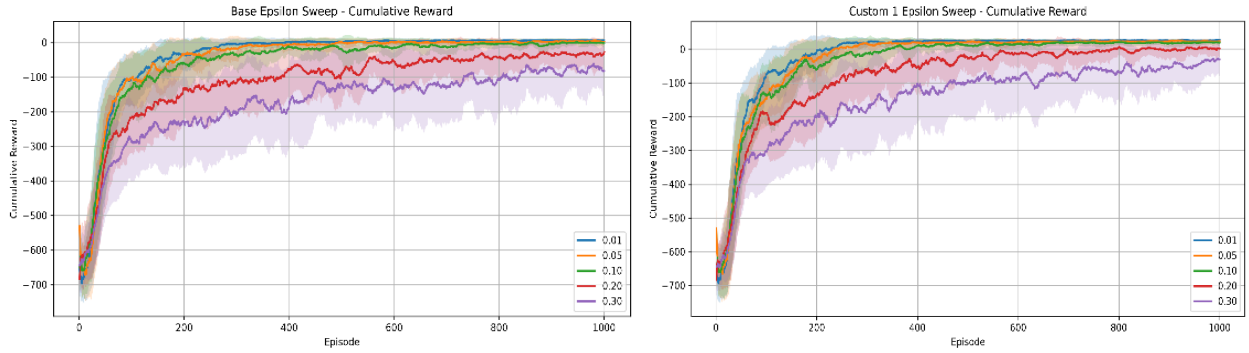
Overall, richer rewards make the system less sensitive to small α values, while higher learning rates—especially α = 0.40—still achieve the best performance.

Key Insights : α = 0.40 selected for the final model: It consistently achieves the fastest and most efficient path minimization across both environments.
• Low α slows convergence: Small step sizes result in slow learning and higher training cost.
• Higher α improves efficiency: Faster episode reduction leads to quicker goal completion and better overall performance.

Conclusion : The analysis shows that higher learning rates significantly improve convergence toward short and efficient policies. Since α = 0.40 consistently achieves the best episode length reduction across both reward settings, it is selected as the optimal final value.

10.6 Hyperparameter (ε) Analysis: Cumulative Reward Comparison

This section examines the effect of the exploration parameter ε (epsilon) on the agent's operational efficiency under both the Base reward and Custom Reward 1 settings. The results measure how different exploration rates influence episode length over 1,000 training episodes.



Overall Trend Across Both Rewards

Across both configurations, a clear relationship emerges between exploration rate and path efficiency. Low exploration values (ε = 0.01–0.05) lead to a rapid and stable decline in episode length, reaching near-optimal performance within ~200 episodes. In contrast, higher values (ε = 0.20–0.30) significantly slow convergence and maintain longer, more variable episodes due to excessive random actions. These effects remain consistent across both reward settings.

Base Reward Analysis (Left Plot)

Under the Base reward setting, episode length is highly sensitive to exploration. Low ε values (0.01–0.05) achieve the fastest reduction from the 200-step cap to a stable baseline of ~15–20 steps with minimal variance. Higher values (especially ε = 0.30) severely hinder convergence, producing unstable and inefficient trajectories even after 1000 episodes. The high level of random action selection continuously disrupts learned optimal paths, making it difficult to stabilize efficient behavior in a sparse reward environment.

Custom Reward 1 Analysis (Right Plot)

Under the Custom Reward 1 (Fast Delivery) setting, all curves show slightly faster early improvement due to denser reward feedback. However, ε = 0.01 still provides the most efficient and stable convergence, achieving the shortest episode lengths. High exploration (ε = 0.30) remains inefficient and volatile, showing that reward shaping cannot compensate for excessive randomness in action selection.

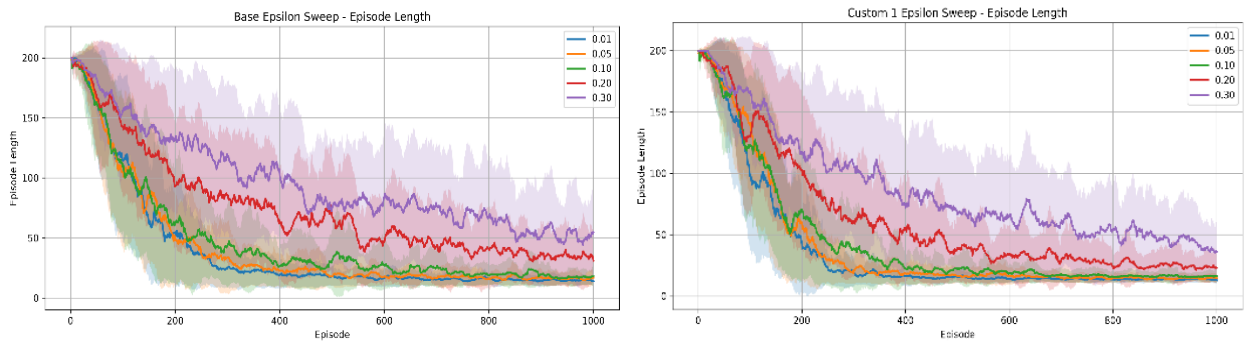
Key Insights

• ε = 0.01 selected for the final model: It consistently minimizes episode length and ensures stable, efficient execution across both environments.
• High exploration introduces operational noise: Excessive random actions prevent stable policy execution and increase episode cost.
• Exploration is most useful early in training: After initial learning, continued high exploration provides no benefit and only degrades performance.

Conclusion : The episode length results confirm that a highly greedy policy (ε = 0.01) is optimal for achieving consistent and minimal path lengths. Across both reward structures, low exploration enables faster convergence and more stable execution, making ε = 0.01 the best configuration for the final model.

10.7 Hyperparameter (ε) Analysis: Episode Length Comparison

This section examines how the exploration parameter **ε** affects episode length under both the **Base Reward** and **Fast Delivery Reward**. Episode length reflects navigation efficiency, with shorter episodes indicating better learned policies.



Overall Analysis

Both reward settings show the same overall behavior. Lower exploration rates (**ε = 0.01–0.05**) reduce episode length much more quickly, allowing the agent to converge to efficient paths early in training. In contrast, higher exploration rates (**ε = 0.20–0.30**) continue introducing random actions after effective policies have been learned, resulting in longer episodes, slower convergence, and greater variance. This demonstrates that the influence of ε is largely independent of the reward function.

Base Reward Analysis

Under the Base Reward, low ε values rapidly decreased episode length and reached stable performance within the first few hundred episodes. Higher exploration rates delayed convergence because the agent repeatedly deviated from efficient routes, leading to longer and more variable episodes throughout training.

Fast Delivery Reward Analysis

A similar trend was observed with the **Fast Delivery Reward**. The additional reward shaping helped reduce episode length slightly faster during the early stages of training, but high exploration rates still prevented the agent from consistently following efficient paths. This indicates that improved rewards cannot fully compensate for excessive exploration.

Key Insights

- **Lower exploration rates (ε = 0.01–0.05)** consistently produced the shortest episodes and fastest convergence.
- **Higher exploration rates** increased both episode length and training variability by encouraging unnecessary random actions.
- The same behavior was observed under both reward functions, confirming that the effect of ε is robust across different reward designs.

Conclusion

The results show that a **small exploration rate** provides the best balance between exploration and exploitation. Once sufficient experience has been gathered, excessive exploration becomes detrimental, preventing the agent from consistently exploiting the learned policy and minimizing episode length.

11. Pragmatic Optimizations

Several practical optimizations were introduced during development to reduce experimentation time while maintaining reliable results.

- During development, experiments were first executed using **a single random seed** to speed up debugging and parameter tuning. The final evaluation was then repeated using **5 seeds** with a 95% confidence interval.
- The kernel was limited to the **5 nearest task-equivalent neighbors**. Larger neighborhoods (e.g., 20 neighbors) increased computation and unexpectedly reduced performance by propagating updates to less relevant states.
- Different numbers of planning steps were evaluated. **Five planning steps** provided the best balance between learning performance and computational cost, while larger values produced only marginal improvements.
- Intermediate policies, checkpoints, and experiment results were saved as **CSV files and policy snapshots**, allowing plots and animations to be regenerated without repeating long training runs.
- Initial experiments used **500 training episodes**. After tuning the planning and kernel parameters, learning had not fully converged by episode 500, so the training horizon was increased to **1000 episodes**, resulting in more stable and better-performing policies.

12. Additional Insights

Throughout this project, we observed that improvements in reinforcement learning do not necessarily come from increasing algorithmic complexity. In several cases, carefully designing the reward function produced a greater improvement than adding more sophisticated learning components. This highlights the importance of providing an informative learning signal before attempting to optimize the learning algorithm itself.

Another important observation was that many techniques are highly dependent on proper parameter tuning. Features such as planning depth and kernel-based generalization were not universally beneficial; their effectiveness depended strongly on selecting appropriate parameter values. This emphasizes that experimental evaluation is an essential part of reinforcement

learning, and that algorithmic additions should always be validated empirically rather than assumed to improve performance.

Finally, the project reinforced that different reinforcement learning techniques often address different limitations. Reward shaping improves the quality of the learning signal, planning increases sample efficiency, and generalization allows knowledge to transfer across similar states. When these components are carefully combined, they complement one another and produce better overall performance than any single technique alone.

13. Quick Start

13.1 Project Overview

This project implements a SARSA-based reinforcement learning agent for the Gymnasium Taxi environment. The implementation integrates Prioritized Sweeping, kernel-based generalization, and custom reward design to investigate their impact on learning performance. The project also includes automated experiments, result visualization, and policy animations for analyzing the learned behaviors.

13.2 Project Structure

FThe project is organized into separate modules to simplify development, experimentation, and result analysis.

Core Implementation (src/)

Contains the main implementation of the reinforcement learning system, including the SARSA agent, reward functions, Prioritized Sweeping, kernel-based generalization, environment creation, evaluation metrics, and the experiment configuration (config.py).

Experiments (experiments/)

Contains all executable experiment scripts:

- **reward_comparison.py** – Compares the Base, Sparse, and custom reward designs.
- **parameter_sweeps.py** – Evaluates different hyperparameter values (e.g., α, ε, planning steps, and kernel parameters).
- **ablation_study.py** – Evaluates the contribution of SARSA, Prioritized Sweeping, Kernel-based Generalization, and their combination. This script also saves the trained policies used for policy animations.
- **plot_results.py** – Generates plots for every experiment currently available in results/csv/.

- **report_plots.py** – Generates the final comparison figures used in the report from all completed experiments.
- **policy_animation.py** – Generates GIF animations of the best trained policies and the policy evolution animation.
- **run_all.py** – Executes all experiments sequentially and automatically generates the report figures.

Results (results/)

Stores all generated outputs in separate folders:

- **csv/** – Raw experiment results.
- **plots/** – Individual plots generated from each experiment.
- **comparison_plots/** – Final comparison figures used in the report.
- **policies/** – Saved Q-tables and training checkpoints.
- **animations/** – Generated GIF animations of the trained policies and policy evolution.

Additional Files

- **tests/** – Verification scripts for individual project components.
- **requirements.txt** – Lists all required Python libraries and versions.
- **README.md** – Provides installation instructions and explains how to execute the project and reproduce the reported results.

13.3 Installation

It is recommended to open the project folder in **Visual Studio Code** and use the integrated terminal to execute the following commands.

1. Create a Python virtual environment:

   python -m venv .venv

2. Activate the virtual environment:

   .venv\Scripts\activate

   After activation, the terminal prompt should begin with **(.venv)**.

3. Install all required libraries:

   pip install -r requirements.txt

All required package versions are listed in **requirements.txt**, allowing the project to be reproduced in a clean environment.

13.4 Running the Project

The project configuration is located in **src/config.py**, where all experiment parameters (e.g., number of episodes, number of seeds, learning rate, exploration rate, planning steps, and kernel parameters) can be modified.

For faster verification of the project, the number of training seeds can be reduced from:

NUM_SEEDS = 5

to

NUM_SEEDS = 1

This decreases the execution time by approximately a factor of five while preserving the complete functionality of the project. The experiments presented in this report were performed using **NUM_SEEDS = 5**.

The recommended way to reproduce all experiments and figures presented in this report is to execute:

python -m experiments.run_all

This command automatically:

- Runs the reward comparison experiment.
- Runs the parameter sweep experiments.
- Runs the ablation study.
- Generates all individual plots.
- Generates all comparison plots used in the report.

Individual components of the project can also be executed separately:

- **python -m src.main** – Trains a single reinforcement learning agent.
- **python -m experiments.reward_comparison** – Compares the different reward designs.
- **python -m experiments.parameter_sweeps** – Performs the hyperparameter tuning experiments.
- **python -m experiments.ablation_study** – Evaluates the contribution of Prioritized Sweeping and Kernel-based Generalization.

After executing an individual experiment, the corresponding plots can be generated using:

python -m experiments.plot_results

To generate the comparison figures used throughout this report, all required experiments must first be completed, followed by:

python -m experiments.report_plots

Alternatively, executing **python -m experiments.run_all** performs both steps automatically.

After completing the **ablation study** (or **run_all**), the policy animations can be generated using:

python -m experiments.policy_animation

The generated results are automatically organized into the following folders:

- **results/csv/** – Raw experiment data.
- **results/plots/** – Individual plots generated from each experiment.
- **results/comparison_plots/** – Comparison figures used in this report.
- **results/policies/** – Saved learned policies and training checkpoints.
- **results/animations/** – Generated policy animations.